OpenAirlines

Solutions for Sustainable Aviation | SkyBreathe®

ISAE SUPAERO
Institut Supérieur de l'Aéronautique et de l'Espace

INTERNSHIP THESIS REPORT

Development of an Automated FDR Validation Tool for Data Quality in SkyBreathe®

Submitted by : Ojasvi Shelar
Ojasvi.SHELAR@student.isae-supaero.fr

Internship at:
OpenAirlines, Toulouse

Tutor : Heidy Dallard
heidy.dallard@openairlines.com

Date: November 4, 2025

Artificial Intelligence & Business Transformation
ISAE 2024-2025

Contents

1. Introduction

This professional thesis presents the design and implementation of a data validation tool for Flight Data Recorder (FDR) data at OpenAirlines, a global leader in airline fuel management software and the creator of the SkyBreathe® eco-flying platform. Reliable FDR data is essential for fuel efficiency analytics and operational monitoring; however, variations in data formats, aircraft types, and decoding methods often introduce gaps and inconsistencies that can degrade downstream insights.

Within OpenAirlines, the CS-Tech department serves as the primary technical interface for airline customers, overseeing the integration of diverse data feeds including FDR and verifying that incoming datasets meet the rigorous standards required by SkyBreathe®. While existing checks are thorough, they are largely manual and time-consuming, making them difficult to scale as the diversity and volume of FDR files increase.

This internship, carried out within the CS-Tech department, focused on designing and implementing a Python-based validation tool that automates key quality checks on transformed FDR files. The aim was to streamline the workflow, reduce manual effort, and improve the reliability of data used by SkyBreathe®.

This report details the technical choices, methodology, and results of the project, and discusses the tool's impact on data quality assurance as well as opportunities for further extension and integration within OpenAirlines' data validation processes.

2. Company Overview

OpenAirlines was established in 2006 in Toulouse by Alexandre Feray, with the primary objective of assisting airlines in optimizing their fuel management, operations, and reducing their environmental impact. Since its inception, the company has evolved into a global leader in airline environmental software, with offices in Toulouse, Hong Kong, Miami, and Montreal. During my internship, I was based at the company's headquarters in Toulouse, located at 31 Rue d'Alsace Lorraine.

The company's mission is driven by the recognition that commercial aviation emits approximately 660 million tons of CO_2 annually, equivalent to more than 20,000 kilograms every second. In response to this challenge, OpenAirlines has focused on developing innovative solutions that enable airlines to reduce both operational costs and their environmental footprint.

Following 8 years of research and development, OpenAirlines launched SkyBreathe®, an eco-flying platform that leverages cloud computing, artificial intelligence, and big data analytics. This solution enables airlines to achieve fuel savings and reduce carbon emissions by up to 5%. Today, OpenAirlines is a trusted partner to over 70 airlines worldwide, including both major international carriers and regional operators. The company's mission is to empower airlines to make data-driven decisions that enhance operational efficiency, reduce costs, and promote sustainability.

At the core of OpenAirlines' product portfolio is the SkyBreathe® 360° platform, a comprehensive suite designed to improve fuel efficiency, lower operational expenses, and support environmental sustainability.

The platform is available in several editions (Essential, Scale, Expert, and Ultimate) and is delivered through a Software as a Service (SaaS) model. This subscription-based approach provides airlines with access to software, secure data hosting, ongoing maintenance, customer support, and

regular updates, allowing them to select the configuration that best aligns with their operational needs.

2.1 Main Products

OpenAirlines provides a range of modular solutions, each targeting a specific aspect of airline efficiency and sustainability:

- **SkyBreathe® Analytics:**
  A comprehensive platform for visualizing and interpreting flight data, enabling airlines to monitor fuel consumption and identify actionable trends for operational improvement.
- **MyFuelCoach®:**
  A mobile application designed for pilots, offering personalized feedback and practical recommendations based on individual flight data to encourage fuel-efficient behaviors.
- **SkyBreathe® APM (Aircraft Performance Monitoring):**
  An advanced module that analyzes aircraft and engine performance metrics, assisting maintenance teams in detecting anomalies and planning interventions to ensure fleet reliability.

- **SkyBreathe® OTP (On-Time Performance):**
  A specialized tool for operations teams to manage flight schedules, analyze delays, and optimize turnaround processes, thereby supporting improved on-time performance.
- **SkyBreathe® ATM (Advanced Trajectory Monitoring):**
  A solution focused on the analysis of flight trajectories, enabling airlines to optimize routing strategies and enhance efficiency through data-driven trajectory management.
- **SkyBreathe® OnBoard:**
  An Electronic Flight Bag (EFB) application that delivers real-time, context-aware eco-flying recommendations to pilots during flight, facilitating the adoption of fuel-saving practices with minimal additional workload.

Each product can be implemented independently or as part of the integrated SkyBreathe® ecosystem, providing airlines with scalable tools to advance both operational performance and environmental sustainability.

Figure 1: SkyBreathe® 360° Platform

2.2 Organizational structure

After introducing OpenAirlines and its product suite, it is important to present the company's organizational structure, particularly the teams responsible for delivering, implementing, and supporting these solutions. Understanding the roles and responsibilities within the organization provides essential context for the technical processes described later in this report.

OpenAirlines is structured around several key departments, each overseen by an executive responsible for both strategy and operations. The primary departments include Product, Science, Engineering, Sales, Marketing, and Customer Experience **(CX).** The CX department is focused on ensuring that airlines maximize the value of OpenAirlines' solutions by managing the integration of complex datasets, overseeing technical onboarding, and providing ongoing support. Central to this effort is the Professional Services team, known as **CS-Tech**, which serves as the main technical point of contact for customers.

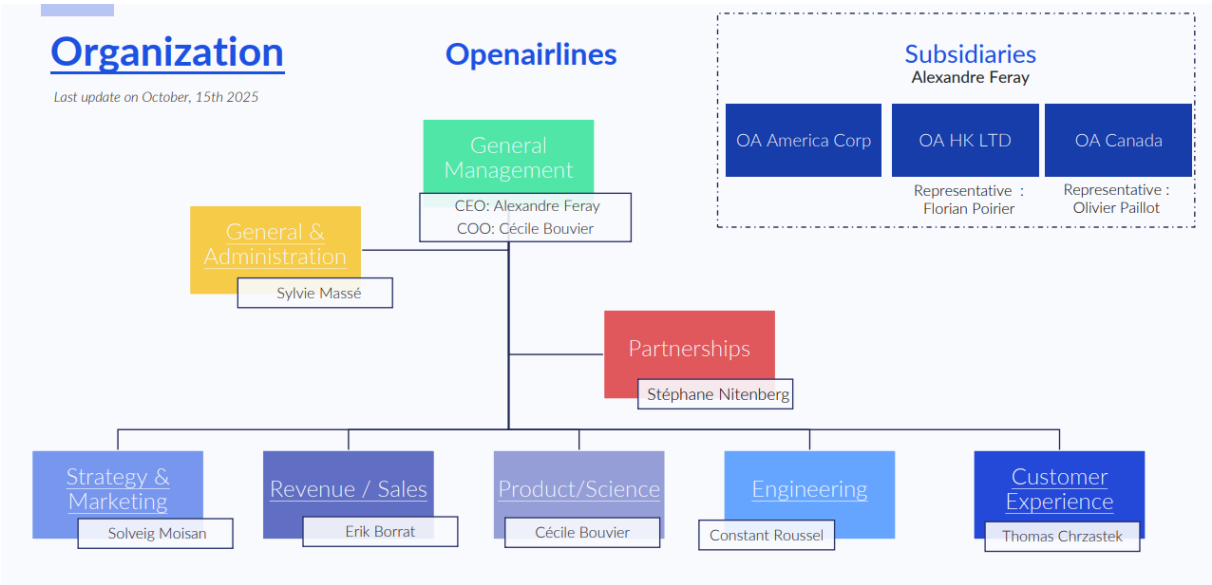


Figure 2: Organization Chart

2.3 The Professional Services (CS-Tech) Team

During my internship, I joined the CS-Tech team within the Customer Experience department. This team is composed of experts specializing in aeronautics, software development, data engineering, project management, data analysis, and data science.

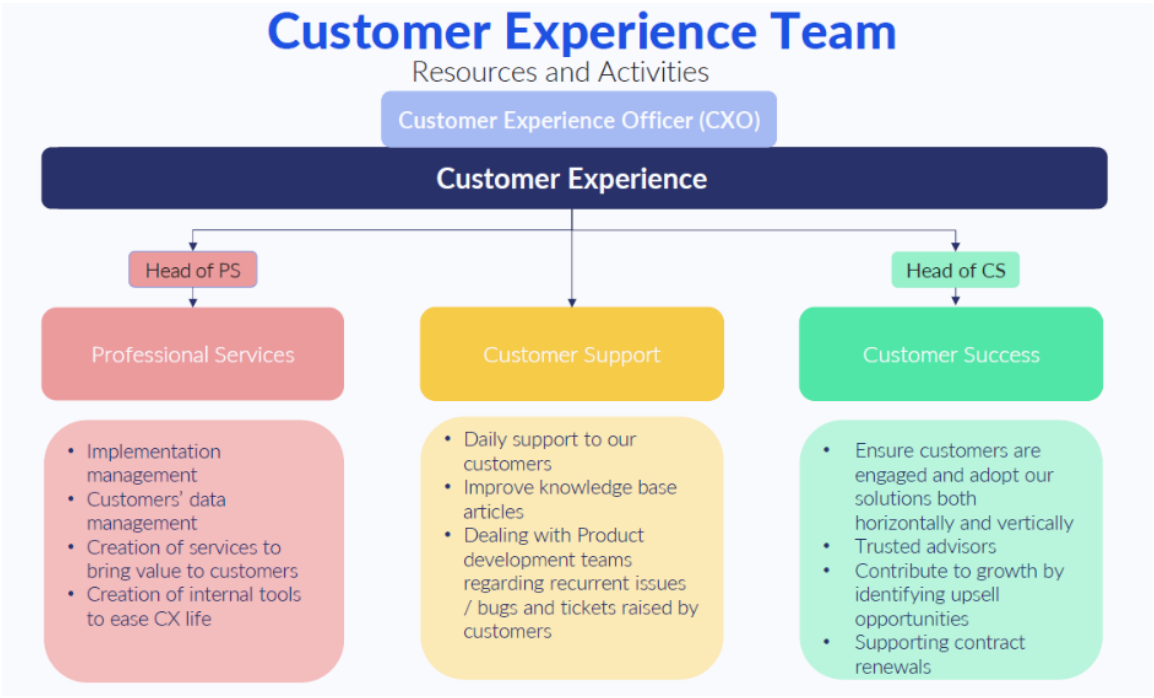


Figure 3: CX Department Chart

As the primary technical interface for airline customers, CS-Tech is responsible for ensuring seamless integration and effective use of OpenAirlines' solutions. Acting as the first technical point of contact, the team's mission can be summarized in three main areas:

- **Data Integration:**

  CS-Tech engineers develop and maintain Python and Java data transformation scripts, known as connectors, to integrate data from airlines. They are also responsible for decoding raw flight data, validating datasets, and creating tools to support data processing and quality assurance.

- **Project Management:**

  The team manages the technical aspects of customer implementation, including project

planning, coordination of weekly meetings, and alignment of internal and external stakeholders. CS-Tech acts as the focal point for technical communication during the implementation phase, ensuring that projects remain on track and that customer requirements are met.

- **Customer Relationship:**

  CS-Tech adapts to the specific needs and constraints of each customer, providing custom data analysis and technical support throughout the implementation process. Once the technical integration is complete, the team ensures a smooth transition to the Customer Success team for ongoing support.

  The structure and responsibilities of the CS-Tech team define the workflow behind OpenAirlines' solutions. The team's multidisciplinary skills are applied across a clear data pipeline, acquiring diverse operational and technical data from airline partners, processing it, and applying quality checks. Each stage addresses specific technical challenges to ensure that the data used by the SkyBreathe® platform is reliable and fit for analysis.

3. Data Processing Workflow

SkyBreathe® operates by collecting and combining several types of operational and technical data provided by airlines. The CS-Tech team is instrumental in overseeing the collection, integration, and processing of these diverse data sources to ensure they effectively support OpenAirlines' solutions. The overall performance of the SkyBreathe® platform depends not only on the quality and consistency of the incoming data, but also on the CS-Tech team's expertise in managing and preparing this data for analysis.

3.1.1 Types of Data

The SkyBreathe® platform utilizes several key types of data, which can be grouped into mandatory and optional categories:

Mandatory Data:

1. **Flight Data Recorder (FDR):**

   FDR data consists of high-frequency technical measurements recorded directly from the aircraft's systems during each flight. These parameters describe the aircraft's behavior, such as speed, altitude, engine performance, and control surface positions.
   FDR data is essential for accurate monitoring of fuel consumption, performance analysis, and anomaly detection. In practice, airlines provide QAR (Quick Access Recorder) or DAR (Digital ACMS Recorder) data, which are like FDR but intended for routine operational analysis rather than accident investigation.

2. **Operational Flight Plan (OFP):**

   The OFP contains the planned route, fuel load, and operational conditions for each flight. It serves as a reference for comparing planned versus actual performance and includes details such as waypoints, alternate airports, and weather forecasts.

3. **Airline Operations Center (AOC):**

   AOC data consists of operational communications sent between the aircraft and the airline's control center. This information encompasses flight planning, scheduling, and execution details, making it essential for tracking daily operations and maintaining coordination between flight crews and ground staff. Additionally, AOC data serves as the main reference for SkyBreathe®, providing the authoritative list of flights.

Optional Data:

In addition to the core datasets, airlines can provide supplementary information to refine analysis and enhance specific features in SkyBreathe.

1. **Loadsheets:**
   Provides information on the aircraft's weight and balance, which improves the accuracy of fuel consumption calculations and performance assessments.

2. **ACARS:**
   Contains additional real-time operational data exchanged between the aircraft and ground systems, such as position reports and maintenance messages.

3. **APU (Auxiliary Power Unit):**
   Records data related to APU ground energy consumption, which can be used to monitor and optimize energy use when the aircraft is on the ground.

4. **Fuel Prices:**
   Information on fuel costs, which can be used for economic analysis and benchmarking.

3.1.2 Raw and Decoded Data

When integrating data into the SkyBreathe® platform, one of the main challenges is handling the different formats and processing levels in which data is received. All flight data are originally created as raw, binary files by the aircraft's recorders. The key question is whether the airline will decode this data themselves or rely on OpenAirlines to handle the decoding.

Raw Data and Binary Files:

In many cases, airlines provide raw data files in binary or hexadecimal formats originating from flight data recorders (FDRs). This raw data contains the recorded flight parameters but is not directly used for analysis. When such files are delivered, OpenAirlines decodes them in-house to convert the binary streams into a structured, readable format.

Recorder types commonly include:

- **DFDR (Digital Flight Data Recorder):** The "black box," recording a wide range of parameters at high frequency. Parameter lists and sampling rates are defined by the aircraft manufacturer in line with regulatory requirements (e.g., EASA, FAA).

- **DAR (Direct Access Recorder):** Configurable by airline and sometimes delivered with a baseline setup from the manufacturer or recorder supplier. Some DAR dataframes are relatively standard and reusable across fleets.
- **QAR (Quick Access Recorder):** Designed for routine operational monitoring and efficiency analysis; captures parameter sets like DFDR but with easier data access.

In short, raw (binary/hex) files must be decoded before they can be used for analysis.

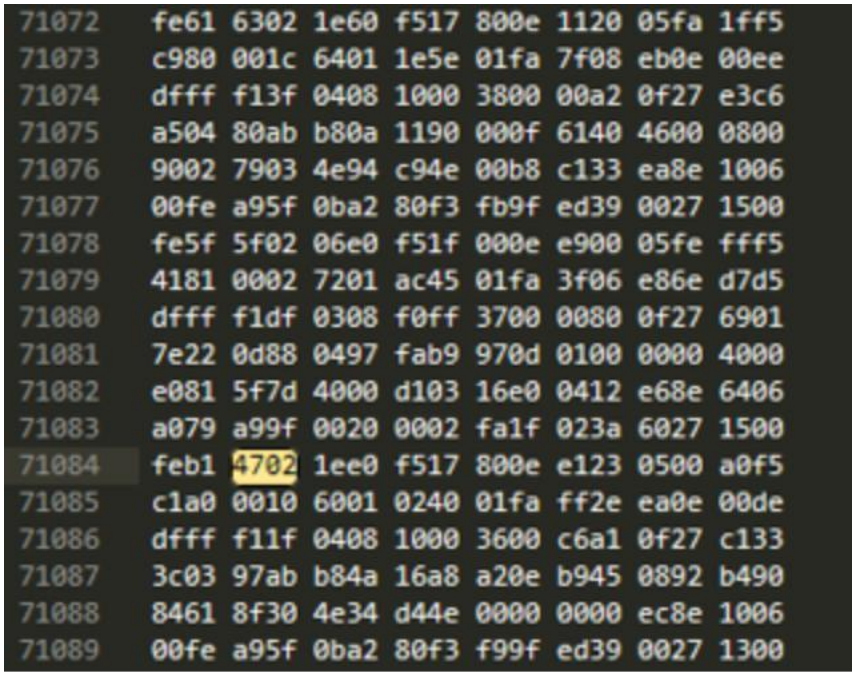


Figure 4: Example of hexadecimal data in a binary file

Decoded Data from Airlines or Providers:

Alternatively, some airlines or their specialized data providers perform the decoding themselves and send OpenAirlines already structured, decoded data, typically in CSV or similar formats, ready for direct integration via the appropriate connector.

Data Ingestion and Mapping

All datasets, regardless of their source or format, are first processed by dedicated software connectors (for FDR, OFP, AOC, etc.), typically written in Python or Java which transform the data into the standardized structure required by SkyBreathe®. These connectors serve as the entry point to the integration workflow, enabling the platform to handle diverse data formats. After transformation, the data is cleansed and checked for quality before being merged and stored in the SkyBreathe® database for further analysis

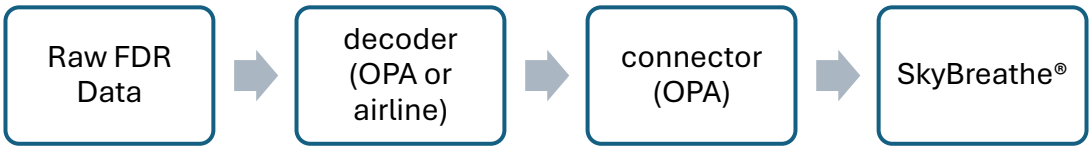


Figure 5: FDR data integration process

4. FDR Data

The main part of my internship at OpenAirlines is to make sure the FDR data is accurate and reliable after it has been converted for use in SkyBreathe®. This includes checking both data received from airlines and data that OpenAirlines has decoded itself.

To understand why this validation is important and sometimes challenging, it is necessary to first look at the nature and complexity of FDR data.

4.1 The Role and Complexity of FDR Data

FDR data stands out as one of the most vital datasets for SkyBreathe®, providing direct technical measurements captured during each flight. Unlike other sources such as OFP or AOC, which primarily offer contextual or operational details, the FDR records hundreds of parameters that comprehensively describe aircraft systems, engine performance, environmental conditions, and flight dynamics.

However, the structure and content of FDR data can differ significantly not only between airlines and aircraft types within the same airline but also based on the specific decoding process and tools used. For example, the same aircraft model may produce FDR files with different dataframes, word-per-second (WPS) rates, or parameter lists depending on whether the data was decoded by the airline, by a third-party provider (such as Ergoss, Aerobytes, or Teledyne), or internally by OpenAirlines. Additionally, the format and quality of the decoded output can be influenced by the type of recorder, the configuration set by the airline or manufacturer, and even the software version or decoding reference used.

To overcome these challenges, OpenAirlines uses a careful standardization process. Since each airline provides FDR data in its own format, the data must first be converted into a single, consistent format before it can be analyzed in SkyBreathe®. This is done through a mapping process, where each airline's specific parameters are matched to standard codes used internally. The mapping is usually defined in an Excel or JSON file, listing all the provider-specific parameter names alongside their SkyBreathe® equivalents. During this step, parameters are grouped into key families, such as:

- **Aircraft Position**: Latitude, Longitude, Altitude, Heading
- **Aircraft Speed:** CAS (Calibrated Airspeed), TAS (True Airspeed), Mach number, etc.
- **Weight:** Center of gravity, fuel in tank, gross weight, etc.
- **Air Information**: outside air temperature, wind direction, wind speed

AOA1,	AOA2,	CAS ,	TAS ,	MACH ,	GS ,	IVV_C,	SPD_SEL,
1.4 ,	0.7 ,	53.5,	0.0,	0.0000,	57,	31 ,	135,
0.7 ,	0.0 ,	60.0,	66.5,	0.0000,	63,	39 ,	135,
1.4 ,	0.7 ,	64.8,	71.0,	0.1010,	68,	45 ,	135,
2.1 ,	1.4 ,	69.3,	75.6,	0.1075,	73,	45 ,	135,
1.8 ,	1.1 ,	74.0,	81.1,	0.1160,	79,	56 ,	135,
2.5 ,	2.1 ,	79.0,	86.0,	0.1215,	84,	73 ,	135,
2.5 ,	2.1 ,	83.0,	89.6,	0.1285,	89,	48 ,	135,
2.8 ,	2.5 ,	88.3,	93.6,	0.1350,	94,	59 ,	135,
2.8 ,	2.5 ,	91.3,	98.1,	0.1425,	99,	59 ,	135,
3.2 ,	2.8 ,	97.0,	103.6,	0.1500,	103,	54 ,	135,
2.8 ,	2.8 ,	101.0,	107.6,	0.1550,	108,	57 ,	135,
3.2 ,	2.8 ,	105.0,	112.9,	0.1630,	113,	60 ,	135,
3.9 ,	3.2 ,	110.0,	117.5,	0.1695,	117,	50 ,	135,
3.2 ,	3.2 ,	115.1,	122.1,	0.1770,	122,	51 ,	135,
4.2 ,	3.5 ,	118.3,	126.1,	0.1830,	126,	58 ,	135,
3.5 ,	3.5 ,	123.1,	130.0,	0.1895,	130,	54 ,	135,
3.9 ,	3.9 ,	126.8,	134.5,	0.1950,	134,	56 ,	135,

Figure 6: Decoded FDR Extract

Some of these parameters are mandatory for core computations in SkyBreathe®, while others are optional and serve to enrich the analysis or enable advanced modules. For instance, the Aircraft Performance Monitoring (APM) module relies extensively on detailed engine and surface control parameters to assess aircraft performance at a granular level

	A	B	C	D	E	F	G
1							
2	**Parameter Code**	**Usual names, possible replacement**	**Status (Analytics)**	**Remarks**	**Usage**	**Provider Field - Dataframe 1**	**Provider Field - Dataframe 2**
3	TAIL_NUMBER	tail, registration	**Mandatory**	**May be in the file name**	Flight identification	File Name	File Name
4	DEPARTURE_DATETIME		**Mandatory**	**Actual departure date and time without seconds ; May be in the file name**	Flight identification	DATE_DAY, DATE_MONTH, DATE_YEAR_C	DATE_DAY, DATE_MONTH, DATE_YEAR_C
5	FLIGHT_NUMBER		**Mandatory**	**In the file name (map header to "cs=" if call sign is provided instead of flight number)**	Flight identification	FLT_NO	FLT_NO
6	ORIGIN		Optional	IATA or ICAO code of airport ; May be in the file name	Flight identification	ICAO_DEPARTURE	ICAO_DEPARTURE
7	DESTINATION		Optional	IATA or ICAO code of airport ; May be in the file name	Flight identification	ICAO_ARRIVAL	ICAO_ARRIVAL

Figure 7: Extract from generic excel mapping for FDR

4.2 The Role of the Python Connector in FDR Data Transformation

A crucial step in ensuring the quality and usability of FDR data within SkyBreathe® is the transformation of decoded airline datasets into a standardized internal format. This transformation is handled by the Python Connector, a tool developed and maintained by the CS-Tech team. The connector is designed to address significant variability in FDR data structures, naming conventions, and record frequencies across different airlines and providers.

The effectiveness of subsequent validation and quality assessment central to my internship depends on the reliability and consistency of this transformation process. Below, I outline the main functions of the connector and how they contribute to producing standardized, analyzable data.

1. Parameter Mapping

One of the primary tasks of the connector is to translate provider-specific parameter names into the standard terminology used by OpenAirlines. For example, one airline might record brake pressure as “BRKP,” another as “Brake_pressure,” while SkyBreathe® requires this parameter to be mapped as “BRAKE_PRESSURE.” This mapping process is guided by a reference file (typically Excel or JSON), which lists all parameters from the provider alongside their corresponding OpenAirlines codes and units.

OpenAirlines Code	OpenAirlines Unit	Provider Dataframe 1	Provider Dataframe 2	Provider Unit
TRUE_TRACK	Degree [0–359]	TRKANGTR	TRKANGTR	Degree [-180 to 180]
AUTO_PILOT_STATUS	Bool	AUTOPILOT ENGAGED	AP Selected	Bool
RIGHT_INBOARD_FUEL_TANK	Kg	FQTY4	FQTY2	Lbs

Table 1: Example of parameter mapping across providers

This mapping ensures that, regardless of the original naming or units, the transformed data is consistent and ready for further validation and analysis.

2. Derived Functions

In some cases, essential parameters required by SkyBreathe® are not directly available in the FDR file. The connector addresses this by allowing the creation of Python functions that derive missing values from other recorded parameters. For example, if Outside Air Temperature (OAT) is not present, it can be computed using available parameters such as Mach number and Total Air Temperature (TAT). This capability ensures that all necessary inputs for analysis are present, even if the original dataset is incomplete.

```python
def map_oat(csv, i, getitem, **_kwargs):  aissa.bachiri
    m = float_or_empty(getitem(csv, i, 'Mach Number 1'))
    t = float_or_empty(getitem(csv, i, 'Total Air Temperature'))

    if m and t:
        mach = float(m.strip())
        tat = float(t)
        return str(aeronautics.oat_from_total_airtemperature(mach, tat))
    else:
        return ''
```

Figure 8: Example of a mapping function to compute OAT from Mach and TAT

3. Parsing and Data Structuring

The connector is also responsible for parsing the original FDR files and structuring the data optimally for analysis. Providers may use different separators (such as commas, semicolons, or tabs), and record frequencies can vary significantly. For instance, some files may record data every 0.125 seconds, but actual values only change every full second. The connector applies frequency rules and data alignment logic to ensure consistency and avoid misinterpretation of the data.

Time (secs)	Altitude	Magnetic Heading	Pitch Attitude	Airspeed	Groundspeed
0.000	*-13	*347.7	*D 1.6	*0.0	*0
0.125					
0.250			*D 1.6		
0.375					
0.500			*D 1.6		
0.625					
0.750			*D 1.6		
0.875					
1.000	*-13	*347.7	*D 1.6	*0.0	*0
1.125					
1.250			*D 1.6		
1.375					
1.500			*D 1.6		
1.625					
1.750			*D 1.6		
1.875					
2.000	*-14	*347.7	*D 1.6	*0.0	*0

Figure 9: FDR with different frequency

By standardizing parameter names, units, and data structure, the Python Connector lays out the foundation for reliable validation and quality assessment of FDR data. My internship builds on this foundation by developing automated tools to systematically check the transformed files, ensuring that the data is not only standardized but also accurate and trustworthy for downstream analysis in SkyBreathe®.

LONGITUDE	LATITUDE	ALTITUDE	HEADING	TRUE_TRACK	DRIFT_ANGLE	PITCH
44.82147217	44.82147217	33046.49198	347.2015796	352.0116358	5.976562977	4.21875
44.82284546	44.82284546	33060.93134	347.2015021	352.0108734	5.888672352	4.21875
44.82559204	44.82559204	33075.37071	347.2014246	352.0101098	5.888672352	4.21875
44.82696533	44.82696533	33089.81007	347.2013471	352.0093449	5.888672352	4.21875
44.82833862	44.82833862	33104.24943	347.2012696	352.0085788	5.888672352	4.21875
44.83108521	44.83108521	33117.68868	347.2011929	352.0078101	5.888672352	4.21875
44.8324585	44.8324585	33131.12792	347.2011162	352.00704	5.888672352	4.21875
44.83520508	44.83520508	33144.56717	347.2010395	352.0062688	5.888672352	4.21875
44.83657837	44.83657837	33158.00642	347.2009628	352.0054962	5.888672352	4.21875
44.83795166	44.83795166	33171.44566	347.2008861	352.0057502	5.888672352	4.21875
44.84069824	44.84069824	33184.88491	347.2008093	352.0060041	5.976562977	4.21875
44.84207153	44.84207153	33198.32416	347.2007325	351.9610602	5.976562977	4.21875
44.84344482	44.84344482	33211.7634	347.2006557	351.9160202	5.976562977	4.21875
44.84619141	44.84619141	33225.20265	347.2005788	351.8708838	5.976562977	4.21875
44.8475647	44.8475647	33238.6419	346.8489394	351.8256509	5.976562977	4.21875
44.84893799	44.84893799	33252.28148	346.848859	351.7803186	5.976562977	4.21875
44.85168457	44.85168457	33265.92106	346.8487786	351.7348892	5.888672352	4.21875
44.85305786	44.85305786	33279.56065	346.8486981	351.6893624	5.888672352	4.21875
44.85443115	44.85443115	33293.20023	346.8486176	351.643738	5.888672352	4.21875
44.85717773	44.85717773	33306.83981	346.8485371	351.6440007	5.888672352	4.21875

Figure 10: Transformed FDR File

5. State of the Art: FDR Data Validation at OpenAirlines

The validation of FDR data at OpenAirlines is a critical step to ensure that the transformed data is accurate, complete, and suitable for analysis within the SkyBreathe® platform. Given the diversity in data formats, aircraft types, and decoding methods, the current validation process is thorough but highly manual, relying on both specialized tools and expert judgment.

5.1 Step-by-Step Validation Process

1. **Preparation and File Selection**

   Validation is performed on files that have already been transformed into the internal SkyBreathe® format. The process typically starts with one file per dataframe (data structure), then expands multiple files and aircraft tails as confidence in the mapping grows.

2. **Manual Inspection Using Internal FDR Viewer**

   - **Visualization:**
     The internal FDR Viewer tool is used to plot flight trajectories and visualize time-series data for all key parameters (e.g., latitude, longitude, altitude, gross weight, OAT, landing gear status).

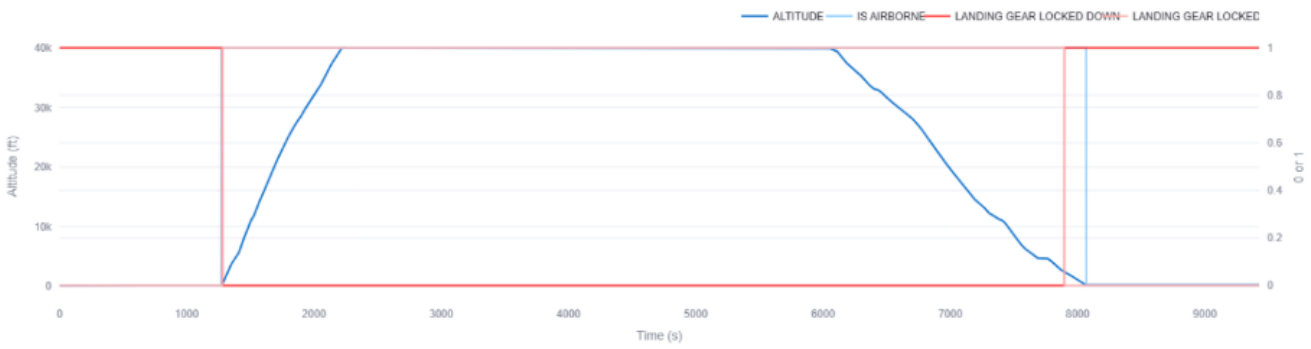


Figure 11: FDR Viewer plot

- **Metadata Review:**

  The tool also allows for inspection of file Metadata, which can help identify missing or inconsistent information.

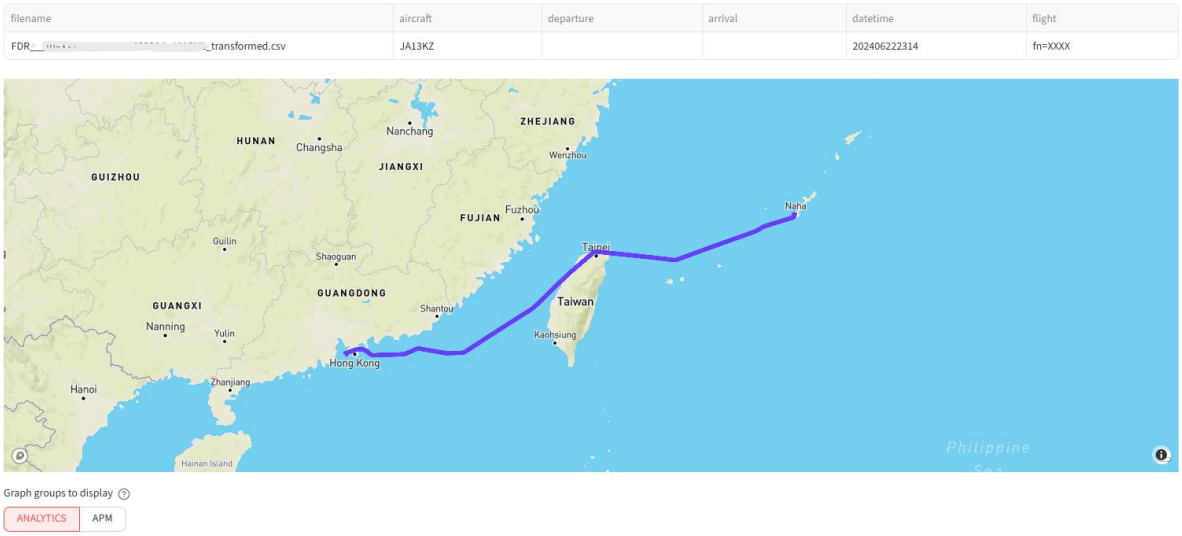


Figure 12: FDR viewer metadata and Trajectory review

3. **Parameter and Consistency Checks**

- **Mandatory Parameters check:**

  Each file is checked to ensure all mandatory parameters are present. If a parameter is missing, its graph will not appear in the FDR viewer.

- **Value Ranges and Behavior:**

  CS-Tech Engineers look for:

a) Out-of-range values (e.g., latitude/longitude spikes).

b) Logical consistency (e.g., landing gear flags should reflect the aircraft's phase of flight).

   Ex. If the aircraft is airborne, "landing gear locked down" should be 0 and "locked up" should be 1.

c) Expected trends (e.g., gross weight should decrease during flight; OAT should be negative at cruise).

d) Smoothness and plausibility of curves (e.g., altitude, air speed).

4. **Documentation and Reporting**

- **Validation Checklist:**

  Findings are documented on a dedicated Confluence validation page for each file or dataframe. The checklist includes:

  a) Presence/absence of each parameter.

  b) Notes on any abnormal or missing data.

  c) Categorization of the validation outcome done as:

  - GREAT: All mandatory parameters and features are present.
  - GOOD: All mandatory parameters are present, but some features are missing or abnormal.
  - KO: One or more mandatory parameters are missing, which would prevent successful imports.

- **Examples of Documentation:**
  - "No Slat," "No Center of Gravity," "Gross Weight Erroneous," etc., are noted for each file.
  - Color-coded flags (red, orange, yellow) are used to highlight severity of these findings.

5. **Scaling the Validation**

Step 1:

Validate one file per dataframe in detail, documenting findings for each parameter**.**

Step 2:

Validate around five files per dataframe (ideally from different aircraft tails), focusing on major findings rather than detailing every parameter.

Step 3:

For large-scale validation, import and check over 100 records per tail, using automated tests (e.g., TestLodge) to ensure high success rates at both the connector and import stages.

Step	Number of files	Method	Expected results	Possible outcomes
Step 1	1 per dataframe	Look at all the plots in FDR viewer Check consistency of value range for all phases Check units Detail findings of validation for each parameter	Every mandatory parameter is available Values are consistent with the relative flight phase Values show expected units	**Great**: every mandatory parameter is available, every feature is available **good**: every mandatory parameter is available, some features will be missing, or user may see abnormal value in skybreathe **ko**: missing a mandatory parameter that will break import
Step 2	~5 per dataframe	Step 1 method on all available parameters No need to detail every parameter, just note important findings	Every mandatory parameter is available Values are consistent with the relative flight phase Values show expected units	Same as step 1
Step 3	>100 records per tail	Decode, connect and import records into skybreathe	100% success rate at connector 99-100% success rate at import Testlodge test complete	

Table 2: FDR Validation Summary Documented in Confluence

5.2 Challenges and Limitations

- **Manual Effort:**

  The process is labor-intensive, requiring engineers to visually inspect each parameter and flight phase.

- **Expert Judgment:**

Detecting subtle issues (such as mapping errors or sensor anomalies) relies heavily on the engineer's experience.

- **Risk of Oversight:**

  Manual documentation and reporting increase the risk of missing issues or inconsistencies.

- **Scalability:**

  As the number of dataframes and aircraft increases, the manual approach becomes less sustainable.

In summary:

The current FDR validation process at OpenAirlines is thorough but highly manual, involving detailed visual inspection, parameter checks, and extensive documentation. While this approach ensures a high level of data quality, it is time-consuming and difficult to scale, highlighting the need for more automated and systematic validation solutions in the future.

6. FDR Validation Tool

6.1 Development Approach

After analyzing the existing FDR validation workflow at OpenAirlines, it became clear that although the process is thorough and reliable, it remains highly manual. Engineers spend significant time visually inspecting flight parameters across various tools, verifying completeness, logical behavior, and overall consistency. While this approach ensures data quality, it becomes increasingly difficult to scale as more aircraft types, decoding formats, parameters, and files are introduced.

The objective of my internship was therefore to design an automated and configurable FDR validation tool capable of reproducing the reasoning used by engineers during manual checks, while improving speed, consistency, and traceability. The intention was not to replace human expertise, but to formalize it into a set of clear, reusable validation rules that can be systematically applied to any transformed FDR dataset.

Overview

The tool was designed to process FDR data delivered as CSV files (or ZIP archives containing CSVs), evaluate their quality, and produce reports that help teams quickly understand data health and identify potential improvements. It was implemented in Python, chosen for its strong data-processing ecosystem (notably pandas and NumPy), its smooth integration with Streamlit, and its compatibility with the existing CSV-based data structure. This combination enables both analysis and reporting to be performed within a single environment.

Development followed an iterative approach: ideas were tested on real datasets, trade-offs were reviewed with my mentor Heidy, and the design was refined step by step. The tool is organized around four main components:

1) **Data Ingestion Layer**: Loads provided .csv or .zip FDR files using pandas, producing DataFrames without preprocessing.
2) **Validation Engine**: Applies configurable rules covering data types, parameter availability, ranges, relational consistency, time-series integrity, and phase-aware logic.
3) **Reporting & Aggregation Layer**: Collects findings and generates structured summaries for both single-file and multi-file analyses.
4) **Visualization & User Interface**: Offers an interactive Streamlit dashboard that allows analysts to explore validation results and focus on the most critical issues.

Immersion in the manual workflow

In the first weeks, I took part in validating FDR files for a few airline implementations and built a connector. During this hands-on phase, I spent a lot of time opening large CSVs in VS Code/Excel to spot-check values and used the FDR Viewer to look at trends. While developing the connector, I used these checks to make sure my code was reliable. After the connector was ready, I validated multiple dataframes to confirm the pipeline. I often had to switch between tools, run quick checks, and cross-reference signals and phases.

That's where the pain points became clear:

- **Plots show trends but hide details**
  Plots are great for signal health but long stretches of NULL in an ~8,000-row file is easy to miss; the curve can look fine while values are wrong.
- **Manual spot-checks don't scale**
  Verifying a single parameter across hundreds of columns and many flights is slow and error-prone
- **Relational rules are hard by eye**
  Checking logical dependencies (e.g. Landing gear flags across phases) requires jumping between signals, phases, and time ranges, which is inconsistent when done manually.

Translating team know-how into clear categories

To translate experience into something automatable, I reviewed three inputs: (1) the team's requirement list, (2) the specific cases they care most about, and (3) Confluence notes from previous airline validations (including issues we observed). I combined these with my field notes and grouped everything so it would be easier to automate and explain:

1. **Data type checks (int, bool, float, string)**
   Make sure each parameter has the expected type.
   Example: Boolean-like flags should be **0/1**, not "ON"/"OFF" or "TRUE"/"FALSE" strings.

2. **Parameter availability (present and not entirely null)**
   Check that required parameters exist and aren't 100% empty.
   Example: ALTITUDE marked as mandatory but completely null.

3. **Parameter ranges**
   Verify values stay within plausible limits.
   Example: HEADING ∈ [0, 360]; a value like **−1** or **>360** should be flagged

4. **Relational rules (cross-parameter consistency)**
   Verify logical relationships between parameters.
   example: if landing gear locked down = 1 then landing gear locked up = 0

5. **Time-series integrity (order and basic behavior)**
   Ensure time and trends make sense across rows.
   Example: TIME should be strictly increasing (no 12:03 → 12:02); GROSS WEIGHT should generally decrease during flight.

6. **Phase-aware checks (context matters)**
   Apply different ranges by phase (and where needed, by aircraft group).
   Example: typical fuel flow is higher in TAKE OFF/CLIMB than in CRUISE.

7. **Aeronautical model checks**
   Compare observed values to standard aeronautical models.
   Example: outside air temperature vs the ISA temperature *at the same altitude; flag large differences beyond a small tolerance.*

8. **Other flight-operations logical checks**
   Basic operational expectations
   Example: enough fuel flow to the engine before take-off

Usecases from CS-Tech — Updated just now — Edit

Person	Feedback	Example(s)
	- after the climb, VNAV mode is not "CLB"	
	I want to check that IS_AIRBONE starts at 0, moves to 1, ends at 0	
	I want to check that fuel flows increase significantly a few minutes before IS_AIRBORNE switches to 1	
	I want to check that APU_ON=0 for most of the time when IS_AIRBORNE=1	
	I want to check that APU_ON=1 at least once for segment where IS_AIRBONE=0	This should not trigger a red alert though, just a warning
	I want to check that if APU_FUEL_FLOW > 0 then APU_ON =1	
	Mandatory parameter for best practices should be present	

Figure 13: Requirements from CS tech team

Centralizing rules in a YAML rulebook

Why YAML (and not code)

I first embedded simple checks (types, basic ranges) directly in Python, then tried a large in-code dictionary. Both worked at small scale, but every small threshold change meant editing code again. That's slow and hard to review as a team. FDR data also isn't "one size fits all. The FDR file mixes time fields, position, engine parameters, speeds, temperatures, and navigation flags. Each has its own type, units, and plausible range. Hard coding all of this in the pipeline is fragile. To keep the "truth about the data" in one place and easy to review, I created a YAML rulebook.

How the rulebook is structured

- **Data types:** taken from existing, trusted SkyBreathe® definitions
- **Parameter ranges (e.g., fuel flow):** based on (1) quick exploration of real FDR files, (2) input from the CS Tech and Science teams, and (3) spot checks against public aircraft documentation/specs. This gave base ranges plus phase-specific overrides per aircraft group.
- **Ranges depend on context (phase).**
  Example: Fuel flow and RPM are low in TAXI, high in TAKE OFF/CLIMB, and stable in CRUISE. A single global range would over-flag or miss real issues.
- **Ranges depend on aircraft family.**
  Example: Turboprops and narrow bodies don't share the same typical fuel flow band in cruise. Without aircraft groups the validation will produce false positives.

With YAML, rules are plain text, teammates can read, question, and edit them. This separation keeps the system reviewable, reproducible, and easy to evolve.

rules.yml

```yaml
rules:
  phases:
    CRUISE:
      FUEL_FLOW:
        margins: { rel_tol: 0.15, abs_tol: null }
        by_group:
          turboprop_aircraft:      { range: { min: 250,  max: 1500 } }
          quadrijet_aircraft:      { range: { min: 6200, max: 15000 } }
```

Figure 14: Parameter ranges defined by phase and aircraft types in yml file

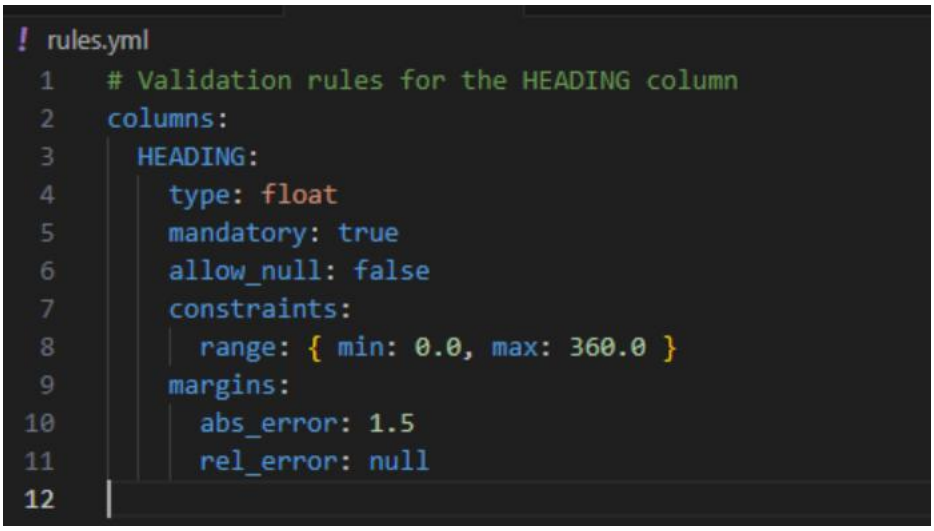


Figure 15: Ex. Parameter rules defined in yaml file

In practice, this makes day-to-day work simpler, we can adjust rules without redeploying the pipeline, and teammates can review a single readable file. As the fleet and datasets grow, we can add aircraft groups or phase ranges directly to YAML. This will keep validation closer to real flight behavior.

Splitting the data by phases

FDR files do not arrive with phases labeled, so I first needed to decide which rows belong to which phase. Phase labels are important because many checks (like fuel-flow ranges) only make sense within a phase.

I followed the general idea used in SkyBreathe® for phase detection but kept it lighter, as SkyBreathe® applies preprocessing (e.g., time fixes) before detecting phases, while I avoided heavy preprocessing to keep scope under control. Given the limited internship timeline, I prioritized a working, phase-aware validation flow using lightweight phase labelling, which is sufficient for relative checks now, with scope for improvement later.

Since not all parameters are always perfect, I relied on a minimal, robust set of signals that are usually present such as GROUND SPEED, ALTITUDE trend, LANDING GEAR STATUS, and AIRBORNE STATUS.

This produces a simple phase order:

GROUND → TAXI_OUT → TAKE_OFF → CLIMB ↔ CRUISE ↔ DESCENT → LANDING → TAXI_IN

I then wrote the detected phase to a new FLIGHT_PHASE column in the pandas DataFrame, which the validators use to apply phase-aware checks and can also be used to plot the flight trajectory for a quick view of the flight.

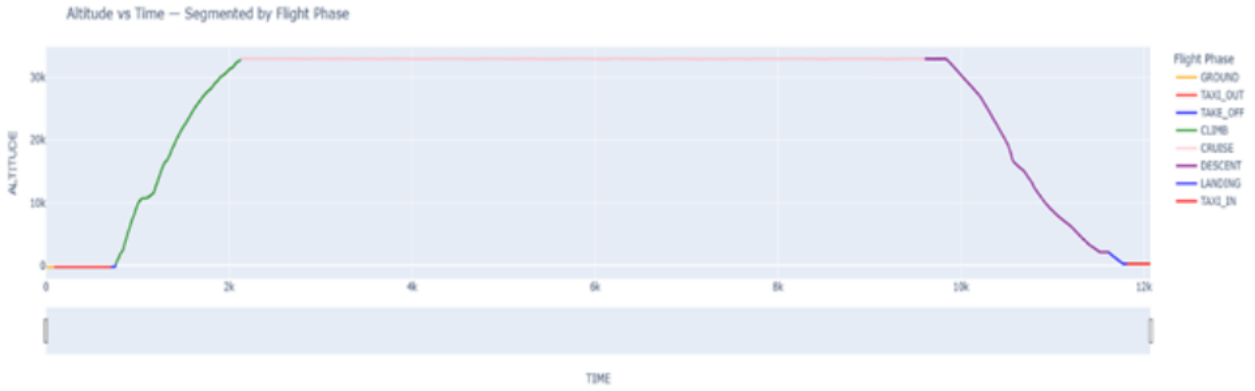


Figure 16: Altitude vs Time (Segmented by Flight Phase)

Turning categories into validator functions (algorithm of each check)

With the YAML rulebook in place and phases defined translated each check category into its own small algorithm, referred to as a *validator*. All validators follow the same general flow:

1. **Load constraints from YAML**: Expected type, mandatory flag, global ranges, and any phase/aircraft-specific ranges.
2. **Select data to evaluate:** Choose the parameters from the DataFrame; use the FLIGHT_PHASE column for phase-based checks.
3. **Evaluate the data quality condition (Validators):** The validator applies the condition to the data. Note that the FDR file data is 1Hz, it means that each row signifies a second
4. **Summarize what failed:** Record the data quality violations with an issue code, how long the issue persisted (time), and a few example values and rows to locate it in the file.
5. **Write findings** for the report. (Json file)

This keeps each validator focused and easy to review

Categorize Severity (ERROR / WARNING / INFO)

Not all findings have the same operational impact. FDR typically starts recording on the ground (before taxi, during pre-flight and engine start). Brief spikes at this stage are expected because (i) some sensors and systems are still initializing, and (ii) some parameters are not valid until the

engines are running or the aircraft is moving. These short ground-phase transients matter less than sustained in-flight anomalies. Likewise, a mandatory parameter not recorded is more critical than an optional one being empty.

To keep results readable, each finding is labeled with severity:

- **ERROR or STRICT ERROR**: Serious issues. Data quality is poor, or a fundamental assumption is broken. This can cause the flight to be rejected by SkyBreathe® or make downstream metrics unreliable.
- **WARNING**: Out of expected limits or suspicious behavior; the file remains usable (possibly after preprocessing), but the issue needs attention.
- **INFO**: Useful context or minor oddities; not a defect.

How severity is decided (technical criteria)

- **Operational/configuration context:** Findings that contradict aircraft configuration logic (e.g., control-surface deflections or landing-gear states inconsistent with the phase) are treated as more severe
- **Reporter escalation:**
    - Magnitude: How far the value exceeds limits,
    - Proportion: How much of the recording/flight is affected,
    - Duration: Longest continuous period of the issue.
- **Fixed mappings:** Some checks are always ERROR (e.g., mandatory parameter missing, TIME not strictly increasing). Missing/empty optional parameters typically remain INFO.

With this scheme, validators detect and describe violations, and the reporter prioritizes them using phase-aware context. The next step is to persist those prioritized findings in a format that's easy to read, parse, and visualize.

Reporting format (JSON)

I chose JSON for the report output because it's lightweight, human-readable, and maps naturally to key/value structures. This makes it easy to store results in a database and to integrate with web applications.

What the JSON includes

- **Metadata:** Source file name and path, when the report was generated.
- **Summary:** Violation counts by severity (ERROR/WARNING/INFO) to gauge data quality briefly.

- **Findings:** One entry per issue with the issue code, parameter(s), short message, and simple context (e.g., example rows).
- **Tables (optional):** Compact data used by the UI, e.g., expected vs observed ranges for out-of-range parameters.

For the user interface, Streamlit was chosen after discussion with my mentor because it can directly read JSON files and display tables or charts with minimal code. It also allows easy implementation of filters such as parameter, issue code, or severity. Additionally, the team was already familiar with Streamlit UI, as it is also used in the internal FDR Viewer.

In practice, the JSON+Streamlit setup provides a fast and efficient path from validator output to an interactive interface, the user can load the JSON file, view a severity summary, browse findings through filters, and examine supporting tables (such as expected versus observed ranges) without the need for a separate backend.


```
{
  "file": "FDR__transformed.csv",
  "generated_at": "10/23/2025 11.01 AM",
  "summary": { "total": 148, "ERROR": 27, "WARNING": 105, "INFO": 16 },
  "groups": [
    {
      "group": "PHASE_FUEL_FLOW_RANGE_VIOLATION_ERROR",
      "total_columns_with_issues": 1,
      "severity": "ERROR",
      "columns": ["FUEL_FLOW:CRUISE"],
      "messages": ["'FUEL_FLOW' outside range [1200.0, 4500.0] in CRUISE for group 'narrow_body_ceo_aircraft': below min for 2 hours 11min 11 sec (min 461.693
    }
  ],
  "tables": {
    "COLUMN_RANGE_TABLE_ERROR": [
      {"col": "LEFT_SPOILER_ANGLE", "observed_range": [-50.36,0.0], "expected_range": [0.0, 60.0]}
    ]
  }
}
```


Figure 17: FDR Validation tool JSON output

6.2 Validators

Validators are the core components responsible for applying predefined quality rules to FDR data. Each validator focuses on a specific type of check and records its results in a structured JSON report.

Below are examples of these validators, outlining their underlying logic and how they assess FDR data quality to generate corresponding findings in the JSON report.

1. Data type Validator

Logic:

Ensure each parameter matches the type declared in the YAML so that later calculations and rules remain reliable. The validator reads the expected type (integer, float, Boolean, string) from the rulebook and cross-checks it with actual data.

Reported issue codes:

BOOLEAN_VIOLATION, STRING_VIOLATION, {col}_NON_NUMERIC

Example:


```
{
  "group": "BOOLEAN_VIOLATION",
  "total_columns_with_issues": 1,
  "severity": "WARNING",
  "columns": ["AIR_CONDITIONING_PACK1_ON"],
  "messages": ["Boolean column 'AIR_CONDITIONING_PACK1_ON' contains values other
than 0/1: ['ON'] for 1 sec (longest run 1 sec, starts at file row 14)."]
}
```


2. Range Validator

Logic:

Checks whether parameters remain within their defined operational limits, as specified in the YAML rulebook.

For each parameter:

- The validator compares values against the valid range [min, max] (inclusive bounds, with tolerance).
- Identifies any out-of-range points, both within and beyond tolerance.
- Quantifies how long and how often the violations occur.

- Reports the expected and observed ranges, duration of the violation, and representative rows for quick context.

This helps separate brief, harmless deviations from sustained anomalies that indicate real data-quality issues.

Severity:

Findings start as a WARNING and escalate to ERROR if any of the following apply:

- The deviation exceeds the defined absolute or relative tolerance. Typical margins (e.g., ±1.5° for angles) allow small variations without over-flagging sensor noise.
- The issue affects a large portion of the flight or persists for longer than a set duration (e.g., 90 seconds).
- For parameters with no tolerance (such as latitude or longitude), any violation is treated as a STRICT-ERROR.

Example:


```
{
  "group": "RANGE_VIOLATION_ERROR",
  "total_columns_with_issues": 2,
  "severity": "ERROR"
  "columns": ["DRIFT_ANGLE","SLAT"],
  "messages": [
"'DRIFT_ANGLE' outside range [-20.0, 20.0]: above max for 34 sec (max 44.297; longest
run 1 sec starting at file row 418; examples at [file row 422, 426, 430]). Observed file
range [0.0, 44.297]",
"'SLAT' outside range [0.0, 30.0]: above max for 1 sec (max 33; longest run 1 sec
starting at file row 11648). Observed file range [0.0, 33.0].",]
}
```


A similar range logic is applied in phase-related checks reported with the parameter and relative phase (FUEL_FLOW: CRUISE)

Example:

```
{
  "group": "PHASE_FUEL_FLOW_RANGE_VIOLATION_ERROR",
  "total_columns_with_issues": 1,
  "severity": "ERROR"
  "columns": ["FUEL_FLOW:CRUISE"],
  "messages": ["'FUEL_FLOW' outside range [1200.0, 4500.0] in CRUISE for group
'narrow_body_ceo_aircraft': below min for 2 hours 11min 11 sec (min 461.693; longest
run 1 hour 57min 25 sec starting at file row 2673; examples at [file row 1832,
2314]). Observed range in phase [461.693, 1232.007].",]
}
```


3. Binary parameter validator

Logic:

This validator checks parameters that behave like on/off system flags. for example, landing gear position, APU state, or airborne status.

It verifies two main quality rules:

A. BINARY_PATTERN_VIOLATION:

Ensures each binary parameter follows a realistic toggle sequence during the flight.

Examples:

- IS_AIRBORNE: [0 → 1 → 0]: Aircraft goes from ground to airborne and back after landing.
- LANDING_GEAR_LOCKED_DOWN: [1 → 0 → 1]: Gear retracts after takeoff and extends again before landing.
- LANDING_GEAR_LOCKED_UP: [0 → 1 → 0] complement of the above.
- APU_ON: [1 → 0] auxiliary power unit switched off after engines start.

These transitions reflect what normally happens during a flight. Some exceptions can give false positives, for example, during training flights or unstable, windy approaches, pilots might retract and extend the gear several times before completing the landing, so a few extra toggles are acceptable.

B. CROSS_BINARY_VIOLATION:

Checks logical consistency between related binary parameters, such as:

- LANDING_GEAR_LOCKED_DOWN == 1 ⇒ LANDING_GEAR_LOCKED_UP == 0
- IS_AIRBORNE == 1 ⇒ APU_ON == 0

Severity:

Both BINARY_PATTERN_VIOLATION and CROSS_BINARY_VIOLATION are generally treated as WARNINGS, since they decoding issues, data misalignment rather than critical sensor failures.

Example:


```
{
  "group": "BINARY_PATTERN_VIOLATION",
  "total_columns_with_issues": 2,
  "severity": "WARNING",
  "columns": ["LANDING_GEAR_LOCKED_DOWN","LANDING_GEAR_LOCKED_UP"],
  "messages": [
"'LANDING_GEAR_LOCKED_DOWN' expected toggle sequence [1, 0, 1], got [0, 1, 0, 1, 0, 1, 0, 1, ...]; toggles at [file row 12, 16, 28, 32, 36, 48, 52, 60, 76, 84].",
"'LANDING_GEAR_LOCKED_UP' expected toggle sequence [0, 1, 0], got [1, 0, 1, 0, 1, 0, 1, ...]; toggles at [file row 10, 20, 34, 42, 50, 58, 72]."]
```


4. Flight Operations Validators

This group of validators captures high-level operational logic rather than direct signal integrity. They assess whether parameter behaviors align with expected aircraft operation, particularly during critical transitions such as engine start, takeoff, and flight weight evolution. These checks help ensure that the FDR data not only looks valid numerically but also makes aeronautical sense.

A. FUEL_RAMP_BEFORE_LIFTOFF

Logic:

The validator checks that engine fuel flow ramps up appropriately just before liftoff (0-1 transition as shown in fig 18), confirming that the engines are providing thrust before the aircraft becomes airborne

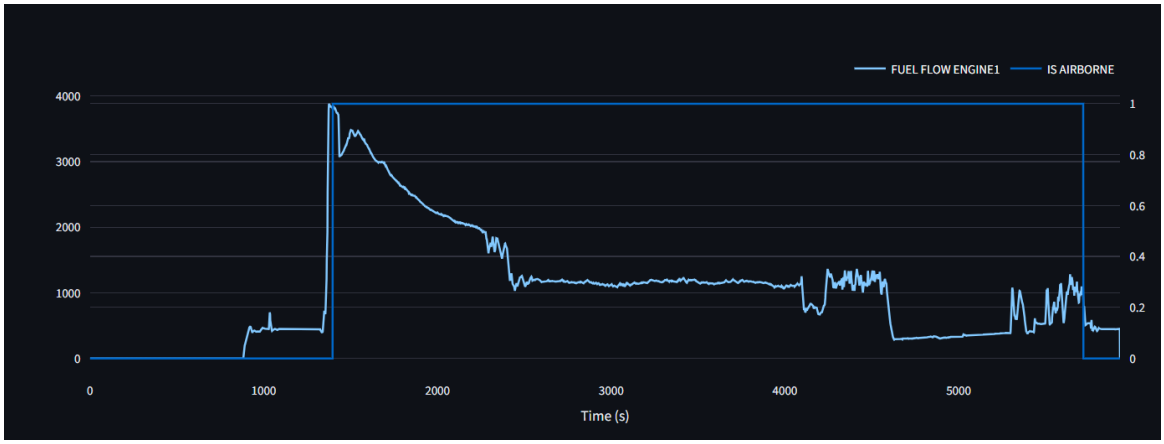


Figure 18: Fuel Flow and Airborne flag vs Time

Severity:

- WARNING when the increase is too weak or flat, suggesting that the liftoff point or fuel signal may be misaligned.
- INFO if the check cannot run because of missing history or missing signals.

This check ensures that the fuel flow profile makes operational sense around take-off. It helps detect cases where FUEL_FLOW is not synchronized with IS_AIRBORNE or when the take-off sequence was not properly captured in the FDR file.

B. ENGINE_SYMMETRY_VIOLATION

Validator checks that paired engines produce similar values during cruise by comparing key parameters like fuel flow, N1, EPR, and EGT. Significant differences can indicate decoding issues.

Severity:

A. **WARNING** when differences persist above the 5% limit.

B. **INFO** if the selected phase or engine parameters are missing.


```
{
  "group": "ENGINE_SYMMETRY_VIOLATION",
  "total_columns_with_issues": 2,
  "severity": "WARNING",
  "columns": ["FUEL_FLOW_ENGINE1","FUEL_FLOW_ENGINE2"],
  "messages": ["Engine symmetry in CRUISE: FUEL_FLOW_ENGINE1 vs FUEL_FLOW_ENGINE2
exceeded 5.0% for 2 hours 11min 11 sec (longest run 2 hours 11min 11 sec starting
at file row 951); worst difference 11.0% at file row 951."
)"]}
```


C. WEIGHT_FUEL_BALANCE_MISMATCH

The validator compares how much the aircraft's gross weight and fuel in tank change over the flight. Since the main contributor to weight reduction during flight is fuel burn, the total decrease in Gross Weight should roughly match the decrease in Fuel in Tank.

Logic:

Compute ΔGW and ΔFuel and if the absolute difference between them exceeds a tolerance (default 5% of the larger change), reports a WEIGHT_FUEL_BALANCE_MISMATCH, stating whether gross weight changed more or less than fuel and by how much.

Severity:

- WARNING when the imbalance exceeds the tolerance, suggesting possible mis-scaling or partial fuel data.
- INFO if parameters are missing

Example:


```
{
  "group": "WEIGHT_FUEL_BALANCE_MISMATCH",
  "total_columns_with_issues": 2,
  "severity": "WARNING",
  "columns": ["GROSS_WEIGHT","FUEL_IN_TANK"],
  "messages": ["GROSS_WEIGHT dropped by 8,854; FUEL_IN_TANK dropped by 4,209: ΔGW ≠ ΔFuel (Δ=-4,645; offset 52.5% vs tolerance 5.0%)"]}
```


D. APU_FUEL_FLOW_CONSISTENCY

Logic:

This validator checks that APU fuel flow is only recorded when the APU is running and flags any rows where fuel flow is present while the APU is off.

Severity:

- WARNING when fuel flow persists while APU_ON = 0, suggesting misalignment or incorrect flag logic.
- INFO if the check cannot run due to missing or invalid data.

5. Time-series Validators

A time-series validator checks rules that depend on how a signal evolves over time

A. GROSS_WEIGHT and FUEL_IN_TANK Behavior Checks

Logic:

These validators monitor the evolution of weight and fuel during the flight to detect unrealistic behavior such as sharp jumps, sudden drops to zero, or unexpected increases (fig. 19)

Severity:

- **WARNING** about all abnormal behaviors (INCREASE, DECREASE, DROPPED_TO_ZERO).
- **INFO** if the parameter or gating signals are missing, making the check inapplicable.

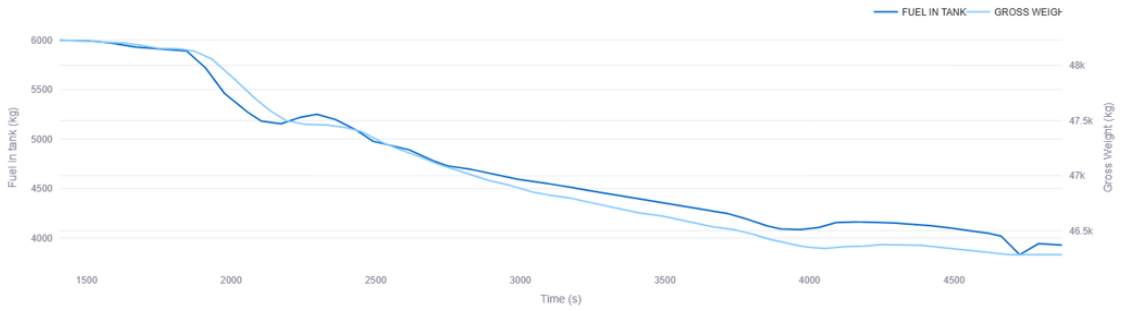


Figure 19: Fuel in tank and Gross Weight vs Time

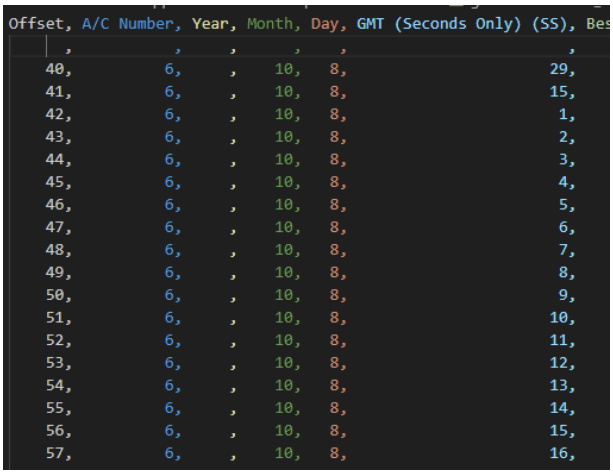
B. TIME monotonicity & gaps

Logic:

Validates that TIME parameter is strictly increasing in a 1 Hz file (one row per second) and that there are no large gaps. These surfaces missing data, decoding issues, or connector mapping problems when timestamps arrive in varying formats (fig 20).

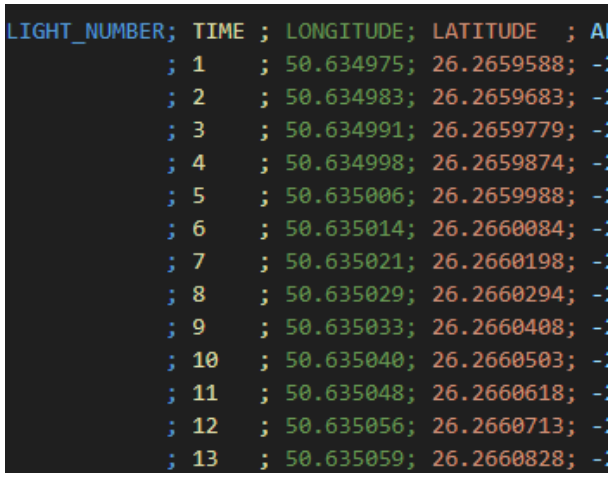
Severity:

ERROR for both TIME_NOT_STRICTLY_INCREASING and TIME_LARGE_GAPS



Offset	A/C Number	Year	Month	Day	GMT (Seconds Only) (SS)	Bes
40,	6,	,	10,	8,	29,	
41,	6,	,	10,	8,	15,	
42,	6,	,	10,	8,	1,	
43,	6,	,	10,	8,	2,	
44,	6,	,	10,	8,	3,	
45,	6,	,	10,	8,	4,	
46,	6,	,	10,	8,	5,	
47,	6,	,	10,	8,	6,	
48,	6,	,	10,	8,	7,	
49,	6,	,	10,	8,	8,	
50,	6,	,	10,	8,	9,	
51,	6,	,	10,	8,	10,	
52,	6,	,	10,	8,	11,	
53,	6,	,	10,	8,	12,	
54,	6,	,	10,	8,	13,	
55,	6,	,	10,	8,	14,	
56,	6,	,	10,	8,	15,	
57,	6,	,	10,	8,	16,	

Figure 20: Time format in received FDR



LIGHT_NUMBER	TIME	LONGITUDE	LATITUDE	A
;	1 ;	50.634975;	26.2659588;	-1
;	2 ;	50.634983;	26.2659683;	-1
;	3 ;	50.634991;	26.2659779;	-1
;	4 ;	50.634998;	26.2659874;	-1
;	5 ;	50.635006;	26.2659988;	-1
;	6 ;	50.635014;	26.2660084;	-1
;	7 ;	50.635021;	26.2660198;	-1
;	8 ;	50.635029;	26.2660294;	-1
;	9 ;	50.635033;	26.2660408;	-1
;	10 ;	50.635040;	26.2660503;	-1
;	11 ;	50.635048;	26.2660618;	-1
;	12 ;	50.635056;	26.2660713;	-1
;	13 ;	50.635059;	26.2660828;	-1

Figure 21: 1Hz Time in transformed FDR

6. Aeronautical Model Validators

These checks assess the consistency, plausibility, and accuracy of flight data by applying aviation-specific rules and expectations. For example, parameters like ground speed, TAS, Mach should be correlated and depend on wind speed, direction, and altitude, while outside air temperature should align with the ISA model.

A. OAT vs ISA

Compares recorded OAT to the ISA temperature at the same altitude (only above ~1,000 ft). Flag rows where |OAT−ISA| exceeds a set tolerance (e.g., ±15 °C).

Severity:

Starts as WARNING (OAT_ISA_WARNING) when deviations exceed the tolerance. Escalates to ERROR when any of the following apply:

- Max deviation ≥2× tolerance
- Proportion of violating rows ≥ 35% of checked rows
- Longest continuous run ≥ 60 s

7. Supplementary Data Quality Validators

- **Phase Summary.**

  Calculates the average duration of each detected flight phase and the average values of key engine or fuel parameters (e.g., fuel flow, torque) to verify that the flight profile is realistic.

- **Empty & Null Counts.**

  Identifies columns that are empty or contain missing values, highlights their importance (mandatory or optional), and detects parameters recorded at periodic intervals.

- **Identical Column Validator.**

  Detects columns that have identical or nearly identical values across the file, which may indicate a connector or mapping error

- **Validate Consecutive Delta (LAT/LONG).**

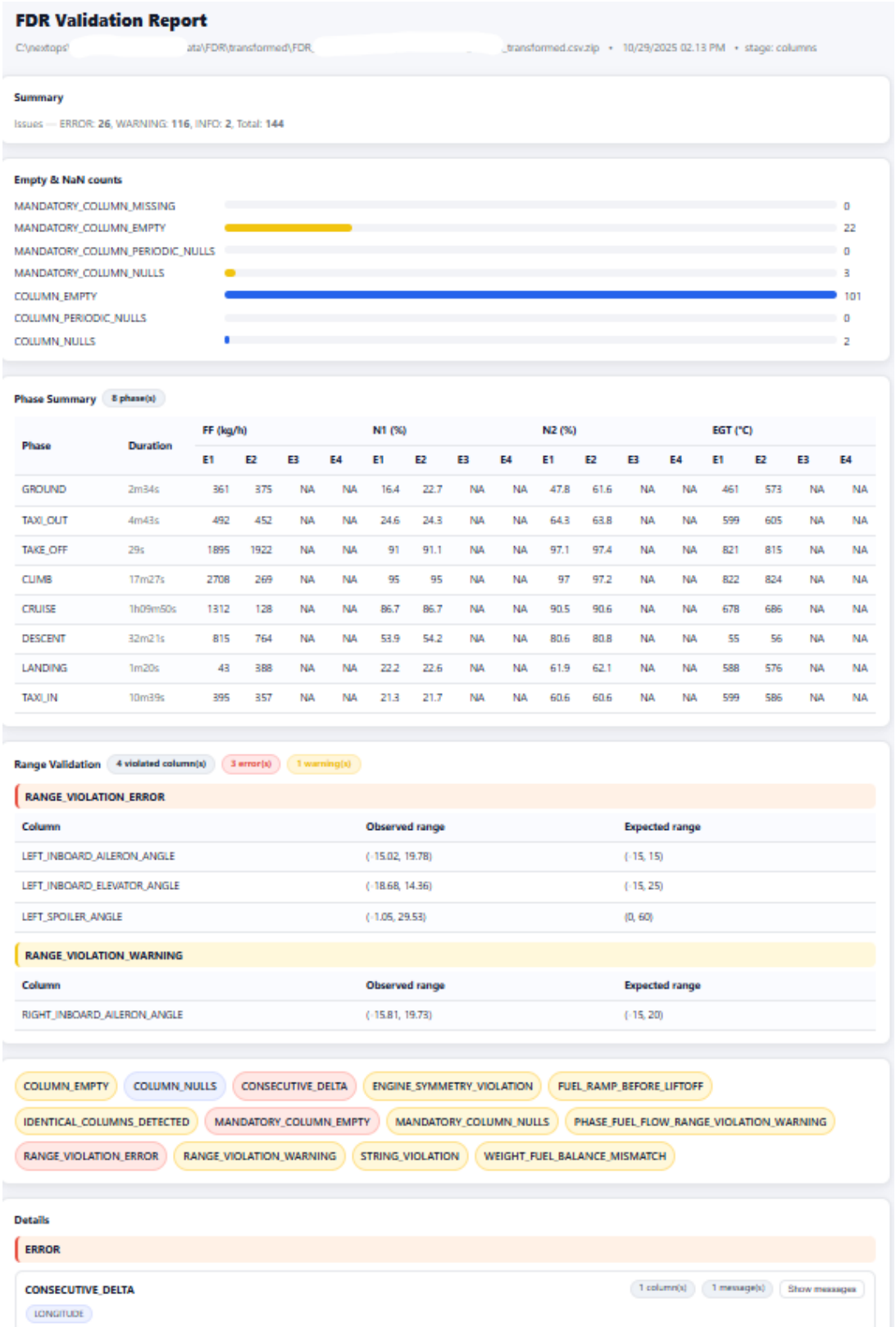
Checks that change between consecutive samples of Latitude and Longitude stay within a small limit (e.g., 0.01 at 1 Hz) and flags sudden jumps that could point to data quality issues.

6.3 Reporting

All validation results are first stored in a structured JSON report, which serves as the primary input for the Streamlit dashboard. As the front-end interface is still under development, a simplified HTML report (fig 22) was implemented for the beta testing phase. This interim format provided a clear and accessible way to review validator outputs, verify the logic of each check, and facilitate feedback during the tool's evaluation.

The HTML report includes the following elements:

- ✓ A bar chart illustrating column availability.
- ✓ A phase summary table.
- ✓ A range violation table.
- ✓ A list of all triggered validators, including column names and detailed messages.



FDR Validation Report

C:\nextops\ ata\FDR\transformed\FDR_ _transformed.csv.zip • 10/29/2025 02.13 PM • stage: columns

Summary

Issues — ERROR: **26**, WARNING: **116**, INFO: **2**, Total: **144**

Empty & NaN counts

MANDATORY_COLUMN_MISSING	0
MANDATORY_COLUMN_EMPTY	22
MANDATORY_COLUMN_PERIODIC_NULLS	0
MANDATORY_COLUMN_NULLS	3
COLUMN_EMPTY	101
COLUMN_PERIODIC_NULLS	0
COLUMN_NULLS	2

Phase Summary

8 phase(s)

Phase	Duration	FF (kg/h)				N1 (%)				N2 (%)				EGT (°C)			
		E1	E2	E3	E4	E1	E2	E3	E4	E1	E2	E3	E4	E1	E2	E3	E4
GROUND	2m34s	361	375	NA	NA	16.4	22.7	NA	NA	47.8	61.6	NA	NA	461	573	NA	NA
TAXI_OUT	4m43s	492	452	NA	NA	24.6	24.3	NA	NA	64.3	63.8	NA	NA	599	605	NA	NA
TAKE_OFF	29s	1895	1922	NA	NA	91	91.1	NA	NA	97.1	97.4	NA	NA	821	815	NA	NA
CLIMB	17m27s	2708	269	NA	NA	95	95	NA	NA	97	97.2	NA	NA	822	824	NA	NA
CRUISE	1h09m50s	1312	128	NA	NA	86.7	86.7	NA	NA	90.5	90.6	NA	NA	678	686	NA	NA
DESCENT	32m21s	815	764	NA	NA	53.9	54.2	NA	NA	80.6	80.8	NA	NA	55	56	NA	NA
LANDING	1m20s	43	388	NA	NA	22.2	22.6	NA	NA	61.9	62.1	NA	NA	588	576	NA	NA
TAXI_IN	10m39s	395	357	NA	NA	21.3	21.7	NA	NA	60.6	60.6	NA	NA	599	586	NA	NA

Range Validation

4 violated column(s) 3 error(s) 1 warning(s)

RANGE_VIOLATION_ERROR

Column	Observed range	Expected range
LEFT_INBOARD_AILERON_ANGLE	(-15.02, 19.78)	(-15, 15)
LEFT_INBOARD_ELEVATOR_ANGLE	(-18.68, 14.36)	(-15, 25)
LEFT_SPOILER_ANGLE	(-1.05, 29.53)	(0, 60)

RANGE_VIOLATION_WARNING

Column	Observed range	Expected range
RIGHT_INBOARD_AILERON_ANGLE	(-15.81, 19.73)	(-15, 20)

COLUMN_EMPTY · COLUMN_NULLS · CONSECUTIVE_DELTA · ENGINE_SYMMETRY_VIOLATION · FUEL_RAMP_BEFORE_LIFTOFF · IDENTICAL_COLUMNS_DETECTED · MANDATORY_COLUMN_EMPTY · MANDATORY_COLUMN_NULLS · PHASE_FUEL_FLOW_RANGE_VIOLATION_WARNING · RANGE_VIOLATION_ERROR · RANGE_VIOLATION_WARNING · STRING_VIOLATION · WEIGHT_FUEL_BALANCE_MISMATCH

Details

ERROR

CONSECUTIVE_DELTA 1 column(s) 1 message(s) Show messages

LONGITUDE

Figure 22: FDR Validation Report on HTML format

6.4 Streamlit web UI (In progress)

A Streamlit web dashboard is being developed to replace the HTML report with a more dynamic, interactive interface. It allows users to visualize validation results for both single-file and multi-file analyses, explore findings by severity or parameter, and quickly identify key data-quality issues.

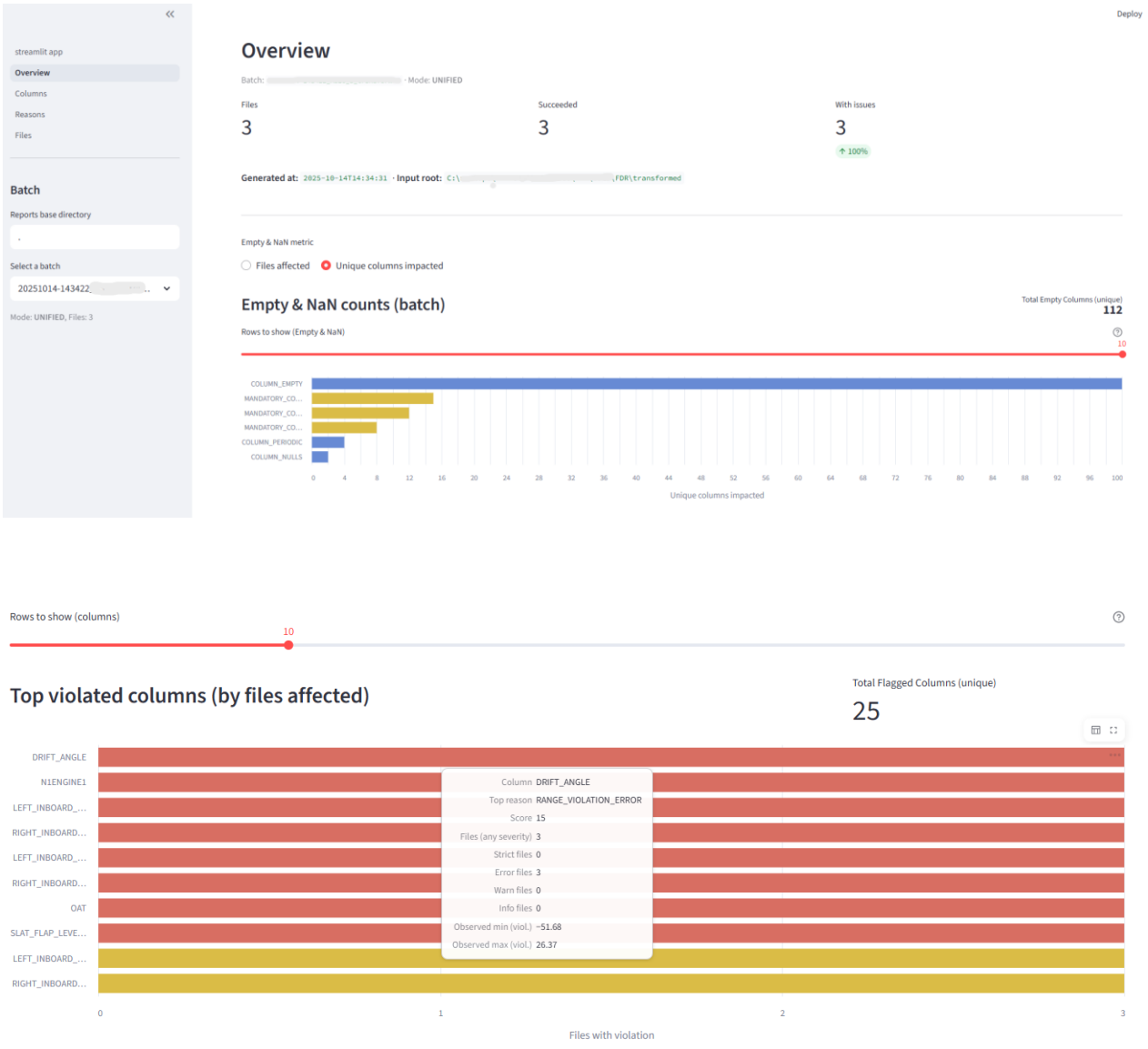


Figure 23: Streamlit Dashboard with multi file mode

7. Results, Feedback and Future Scope

This section presents the main outcomes of the tool's beta testing phase, including cases of real data-quality issues detected by the validators. It also summarizes the feedback received from the CS-Tech team during evaluation and outlines potential improvements and future development directions for the FDR validation tool.

Case 1 – Fuel-flow decoding issue

Although the fuel-flow plots for engines initially appeared correct, the average fuel-flow value during cruise was approximately 400 kg/h (fig 25 left), well below the expected range of [1200 – 4500 kg/h].

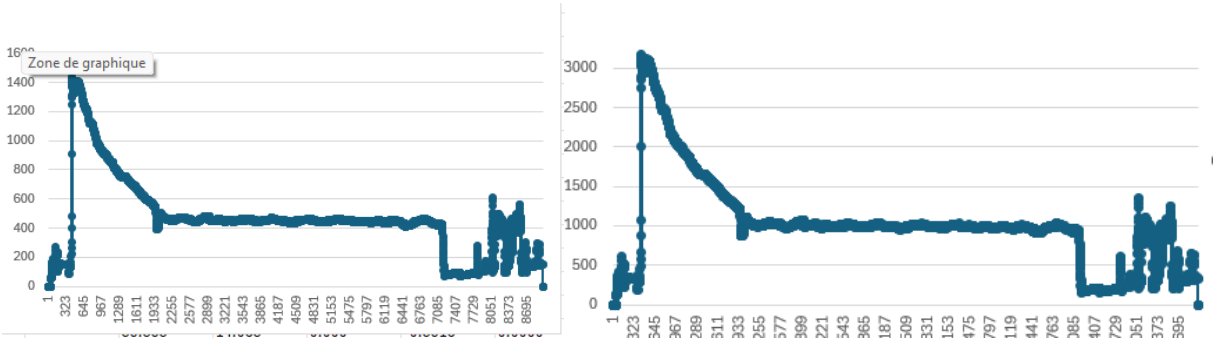
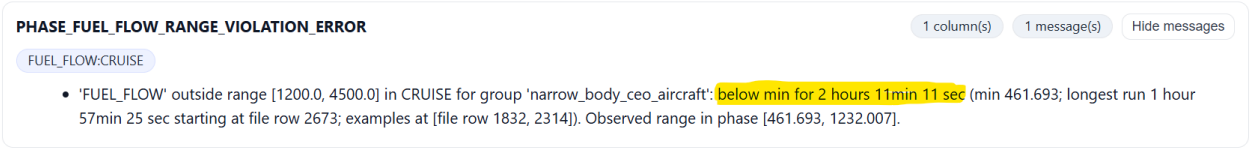


Figure 24: Fuel flow plot before and after correction

This discrepancy could easily have gone unnoticed during manual review, but the *phase-range validator* flagged the anomaly as "fuel flow below expected range during CRUISE." Further investigation revealed a decoding issue in OpenAirlines' internal process, which was subsequently corrected.



Case 2 – Range and decoding inconsistencies

Through the range-violation summary, the team identified abnormal limits for several parameters, including TRUE_TRACK, and informed the airline to harmonize their data exports. For spoiler-angle parameters, the tool also uncovered an underlying error in the decoding

polynomial for A320 NEO aircraft types. Secondary control-surface signals like spoilers are often subtle and easy to overlook during manual checks, making this an especially valuable finding.

Range Validation 8 violated column(s) 8 error(s)

RANGE_VIOLATION_ERROR

Column	Observed range	Expected range
TRUE_TRACK	(-179, 179.7)	(0, 360)
LEFT_OUTBOARD_AILERON_ANGLE	(-23.64, 28.48)	(-25, 25)
RIGHT_OUTBOARD_AILERON_ANGLE	(-24.61, 28.21)	(-25, 25)
LEFT_OUTBOARD_ELEVATOR_ANGLE	(-30.9, 17.4)	(-20, 25)
RIGHT_OUTBOARD_ELEVATOR_ANGLE	(-30.4, 18.4)	(-20, 25)
LEFT_SPOILER_ANGLE	(-50.36, 0)	(0, 60)
RIGHT_SPOILER_ANGLE	(-50.4, 0)	(0, 60)

Critical Feedback and Areas for Refinement

- **Phase detection accuracy:**

  The current phase identification logic needs improvement to reach the same level of reliability as the one used in SkyBreathe®.

- **Range definition precision:**

  Some range limits defined in the YAML configuration were found to be either too broad or too strict. These should be reviewed with the Science Department to better reflect real operational values for each aircraft type

- **Validator logic improvements:**

  Certain checks, such as *Fuel Flow Ramp Before Liftoff*, did not always trigger as expected. Adjusting their logic and thresholds will make these validators more consistent and reliable.

- **Clarity of validation messages:**

  Feedback from the CS-Tech team indicated that some outputs were difficult to interpret. Adding short descriptions or clearer explanations for each validator code would make the results easier to understand and more actionable.

Future Scope

Looking ahead, several developments can further enhance the performance, usability, and analytical depth of the FDR validation tool:

- **Performance optimization:**

  Improve the processing speed and overall efficiency of the tool to handle larger datasets and multi-file analyses more quickly.

- **Enhanced Streamlit dashboard:**

  The current dashboard already supports both single-file and multi-file modes. Future improvements will focus on refining the multi-file interface to provide a clearer and more intuitive overview of overall data quality across all customer-provided datasets.

- **Expansion of validation coverage:**

  Add new, specialized validators to cover additional aircraft systems and operational behaviors, thereby increasing the scope and robustness of quality checks.

- **Integration with visualization tools:**

  Develop an interactive Streamlit dashboard connected to the internal FDR Viewer, enabling users to select parameters, visualize time-series plots, and view detected issues simultaneously.

- **Aircraft and engine modeling:**

  Extend YAML rule definitions to include engine types and corresponding performance ranges alongside aircraft models, ensuring more precise and context-aware validation.

8. Conclusion

This internship at OpenAirlines offered a unique opportunity to apply data engineering and analytical methods to real-world aviation data. Through the development of the **FDR validation tool**, I gained hands-on experience in building an end-to-end software solution from designing validation logic and structuring algorithms to implementing reporting workflows and visualization layers.

At the start, I was somewhat concerned about my limited background in software development. However, with constant guidance from my mentor and support from the CS-Tech team, I was able to progressively strengthen my programming and architectural skills. I learned to design modular Python code, organize validators into reusable components, and manage data pipelines efficiently using pandas, NumPy, and Streamlit.

Working with large FDR datasets also gave me practical experience in handling noisy time-series data, applying data-cleaning strategies, and developing algorithms for phase detection, range validation, and behavior analysis. I learned to balance computational performance with interpretability, optimizing validation speed while ensuring results remain transparent and auditable.

On the data-analysis side, I deepened my understanding of aircraft operational parameters, fuel efficiency metrics, and how data inconsistencies can affect downstream analytics in the SkyBreathe® platform. Collaborating with engineers and data scientists also improved my ability to interpret aeronautical data within a business and operational context.

Beyond technical growth, this internship taught me how to translate manual engineering checks into automated, explainable logic, bridging the gap between domain expertise and software design. It was an opportunity to apply my background in **Artificial Intelligence and Business Transformation** to a concrete problem in aviation, contributing to OpenAirlines' broader goal of promoting data-driven and sustainable flight operations.

9. Appendices

Appendix A: Abbreviations

APU	Auxiliary Power Unit
CSV	Comma-Separated Values
CS-Tech	Customer Success – Technical Department
DFDR	Digital Flight Data Recorder
DAR	Direct Access Recorder
QAR	Quick Access Recorder
FDR	Flight Data Recorder
ISA	International Standard Atmosphere
OAT	Outside Air Temperature
TAT	Total Air Temperature
YAML	Yet Another Markup Language
UI	User Interface
JSON	JavaScript Object Notation
SaaS	Software as a Service
KPI	Key Performance Indicator
AI	Artificial Intelligence

Appendix B: Technical Terms and Tools

Term / Tool	Description
Pandas	A Python library used for data manipulation and analysis, particularly suited for handling tabular data in DataFrames.
NumPy	A Python library for numerical computing, providing support for large arrays and mathematical operations.
Streamlit	A Python framework for building interactive web applications and dashboards directly from scripts, used for the tool's interface.
YAML Rule File	A configuration file that defines expected data types, valid ranges, and validation rules for each FDR parameter.
JSON Report	The structured output format of the validators, containing detailed results and metadata for each check.
Validator	The core part of the tool is responsible for applying quality rules, detecting anomalies, and generating findings.
DataFrame	A data structure in Pandas that organizes data into labeled rows and columns, like a spreadsheet or SQL table. In the context of this project, DataFrame also refers to an individual dataset provided by airlines containing recorded flight parameters.
Connector	The component that integrates customer-provided data into OpenAirlines' internal format for analysis in SkyBreathe®.
Phase-Range Validator	A specialized validator that compares parameter values (like fuel flow) against expected limits for specific flight phases.
Confluence	The documentation platform used by OpenAirlines' to record technical processes, product specifications, and operational guidelines for internal collaboration.
FDR Viewer	An internal visualization tool at OpenAirlines for exploring flight data and comparing recorded signals.